



- ☑ 본 과제는 학회 정규 세션 「Intro to DL」, 「CNN」, 「LLM Basic」, 「LLM with Transformer」의 내용을 다루며, 개념의 적용과 실제 활용 사례에 대한 이해를 돕기 위해 기획되었습니다. 해당 과제는 평가를 위한 것이 아니므로 세션 자료를 참고하고, 학회원 간 토론 및 Slack 질의응답을 활용하여 해결해 주십시오. 단, 답안 표절이나 LLM의 남용은 금지합니다.
- ☑ 8/23 (일) 23 시 59 분까지 GitHub 에 PDF 파일과 코드 문제 파일을 하나의 ZIP 파일로 묶어 제출해 주십시오. 제출 과정에서 어려움이 발생하면 15 기 학술부 (안재민, 이은민, 현승원) 로 문의해 주십시오.

문제 1 Forward-Backward Propagation

신경망 학습은 순전파 (forward propagation) 와 역전파 (backpropagation) 의 반복적인 과정을 통해 이루어집니다. 순전파는 입력값으로 예측 결과를 도출하고 오차를 계산하는 과정이며, 역전파는 이 오차를 기반으로 가중치를 업데이트하는 과정입니다. 이 문제를 통해 두 과정이 어떻게 상호작용하며 신경망이 실제로 학습하는지 그 원리를 직접 확인해 봅시다.

다음과 같은 neural network 가 있다고 가정해 봅시다. 각 weight 의 초기값은

$$w_1 = 0.1, \quad w_2 = 0.2, \quad w_3 = 0.3, \quad w_4 = 0.4, \quad w_5 = 0.5, \quad w_6 = 0.6, \quad w_7 = 0.7$$

입니다.

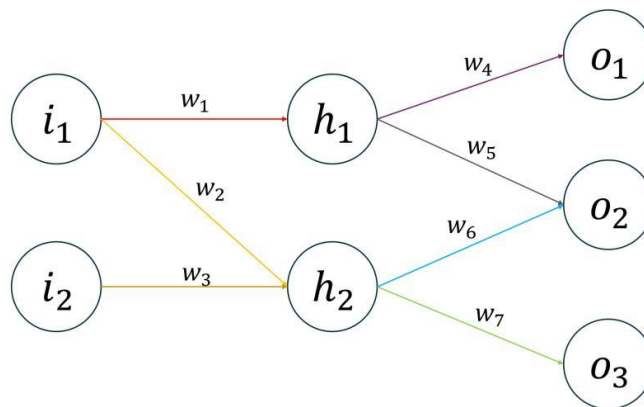
입력값은 $i_1 = 1, i_2 = 2$ 이고, 타겟값은 $y_1 = 0.5, y_2 = 1.5, y_3 = 3$ 입니다.

Hidden layer 의 h_1, h_2 에는 sigmoid function 을 적용하고, output layer 의 o_1, o_2, o_3 에는 별도의 activation function 을 적용하지 않습니다. Bias 는 고려하지 않습니다.

손실 함수는 다음과 같은 Mean Squared Error(MSE) 를 사용합니다.

$$L = \frac{1}{3} [(y_1 - o_1)^2 + (y_2 - o_2)^2 + (y_3 - o_3)^2].$$

학습률은 $\gamma = 0.2$ 이며, 이번 문제에서는 한 번의 순전파와 역전파를 통해 weight 가 어떻게 업데이트되는지 계산합니다.



(위 그림에 화살표로 표시된 연결만 존재합니다. 즉, fully-connected 구조가 아니며, 화살표가 없는 노드 쌍 사이에는 weight 가 존재하지 않습니다.)

1) h_1, h_2, o_1, o_2, o_3 를 각각 계산하고, 주어진 MSE 를 이용하여 Loss 를 계산하세요.

$$h_1 = \sigma(w_1 \cdot x_1) = \sigma(0.1)$$

$$o_1 = w_4 \cdot h_1$$

$$h_2 = \sigma(0.8)$$

$$o_2 = w_5 \cdot h_1$$

2) $\frac{\partial L}{\partial w_4}$ 를 계산하세요.

$$= \frac{2}{3} \times (o_1 - y_1) \cdot h_1$$

3) $\frac{\partial L}{\partial w_1}$ 을 계산하세요. Chain Rule .

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial h_1} \cdot \frac{\partial h_1}{\partial w_1}$$

4) 2), 3) 의 결과를 이용하여 한 번의 역전파를 통해 w_1, w_4 를 업데이트하세요.

시그모이드 함수는 $\sigma(x) = \frac{1}{1 + e^{-x}}$, $\frac{d}{dx} \sigma(x) = \sigma(x)(1 - \sigma(x))$ 이며, gradient descent 에서는 $w \leftarrow w - \gamma \frac{\partial L}{\partial w}$ 로 업데이트합니다.

$$w_4 = 0.4 - 0.2(-0.1015)$$

$$w_1 = 0.1 - 0.2(-0.08114)$$

문제 2 CNN

CNN 은 이미지와 같은 고차원 데이터를 효과적으로 처리하기 위해 설계된 신경망 구조로, 합성곱 (Convolution) 연산을 통해 국소적인 패턴을 추출합니다. 이번 과제에서는 주어진 CNN 구조를 기반으로 순전파 계산, 필터가 감지하는 패턴의 해석, MLP 와의 비교, 그리고 역전파를 통한 가중치 학습 과정을 단계별로 살펴봄으로써 CNN 의 기본 동작 원리를 이해해 봅시다.

다음과 같은 4×4 크기의 grayscale 이미지 x 가 있다고 가정해 봅시다.

$$x = \begin{bmatrix} 1 & 2 & 1 & 0 \\ 0 & 1 & 2 & 2 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \end{bmatrix}$$

CNN 구조는 다음과 같습니다.

Step 1: (Conv1) filter

$$w_1 = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}, \quad \text{stride} = 1, \quad \text{zero-padding} = 1$$

Step 2: (MaxPool) 2×2 Max pooling, stride = 2

Step 3: (Activation function) ReLU

$$f(x) = \max(0, x)$$

Step 4: (Conv2) filter

$$w_2 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad \text{stride} = 1, \quad \text{zero-padding} = 0$$

(Bias 는 고려하지 않으며, convolution 연산에서는 filter 를 뒤집지 않고 그대로 적용합니다.)

1) 주어진 CNN 구조에 대해 입력이 x 일 때 forward propagation 을 계산하세요. Conv1, MaxPool, ReLU, Conv2 각 단계에서 얻어지는 feature map 의 크기와 값을 순서대로 제시하세요. 또한 Step 2 와 Step 3 의 순서를 서로 바꾸어 계산할 경우 최종 결과가 달라지는지 확인하고, 그렇게 되는 이유를 함께 설명하세요.

$\max(\text{ReLU}) = \text{ReLU}(\max)$

$$\begin{bmatrix} -3 & 2 & 1 & 3 \\ -3 & 2 & -1 & 4 \\ -2 & 0 & -3 & 3 \\ -1 & 2 & -2 & 1 \end{bmatrix} \Rightarrow \begin{bmatrix} -2 & 4 \\ 2 & 3 \end{bmatrix} \rightarrow \begin{bmatrix} 0 & 4 \\ 2 & 3 \end{bmatrix}$$

2) Conv1 의 필터 w_1 과 Conv2 의 필터 w_2 는 각각 어떤 종류의 패턴을 감지하는지 설명하고, 1) 에서 얻은 결과를 해석해 보세요.

$\downarrow$ 수평방향 $\downarrow$ 수직방향

3) 위의 CNN 과 동일한 입력을 받는 MLP 를 가정해 봅시다. MLP 는 CNN 보다 표현할 수 있는 함수의 범위가 더 넓음에도 불구하고, 동일한 파라미터 예산에서 이미지 데이터셋에 대한 일반화 성능은 일반적으로 CNN 이 더 우수합니다. 그 이유를 설명하세요.

이공치공 & local connectivity

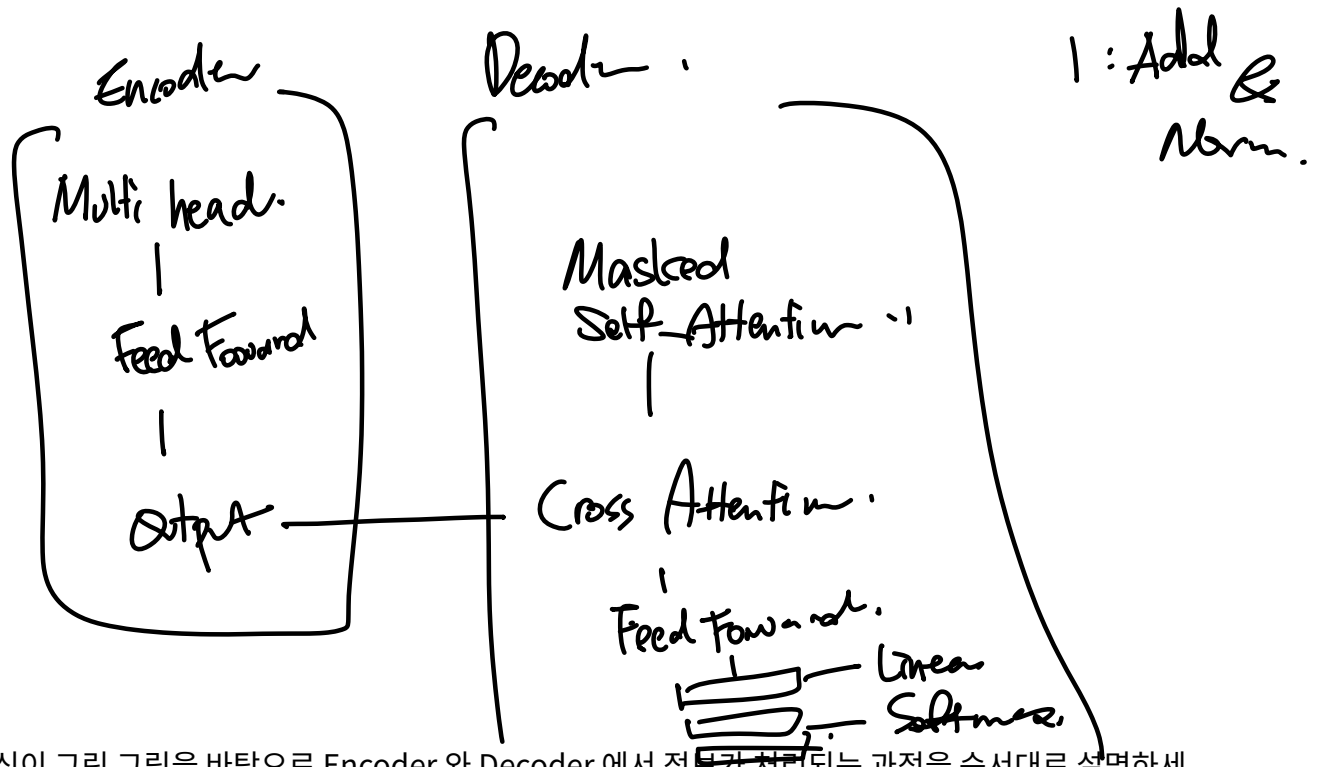
4) 최종 출력에 대한 손실 L 이 주어졌다고 가정할 때, Conv1 의 필터 w_1 을 업데이트하기 위한 Back Propagation 과정에서 chain rule 이 어떻게 적용되는지 설명하세요. Conv2, ReLU, MaxPool 을 거쳐 Conv1 까지 gradient 가 전달되는 흐름을 중심으로 설명하되, 필터 gradient 의 구체적인 수치 계산 과정은 제시하지 않아도 됩니다.

$L \rightarrow \text{가공방배} \rightarrow \text{ReLU} \rightarrow \text{Max Pool} \rightarrow \text{Conv}$

문제 3 Transformer Architecture

Transformer 는 recurrence 나 convolution 없이 attention 을 중심으로 입력 토큰 사이의 관계를 학습하는 모델입니다. 이번 문제에서는 Transformer 의 전체 구조를 직접 그려 보고, 각 구성 요소를 따라 데이터가 어떻게 이동하며 다음 토큰의 확률 분포가 생성되는지 설명해 봅시다.

1) 강의에서 다룬 표준 Encoder-Decoder Transformer 의 전체 구조를 직접 그려보세요. Transformer 아키텍처의 흐름을 이해하기 위함이니 흐름을 생각하시며 그려보세요.



2) 자신이 그린 그림을 바탕으로 Encoder 와 Decoder 에서 정보가 처리되는 과정을 순서대로 설명하세요. 특히 Encoder 의 출력이 Decoder 의 어느 부분에서, 어떤 목적으로 사용되는지 서술하고, Decoder 의 최종 출력이 어떤 과정을 거쳐 다음 토큰에 대한 확률 분포로 변환되는지도 함께 설명하세요.

Encoder..



입력 토큰 벡터 $\Rightarrow$ Decoder 에서 Key-Value .

$\Rightarrow$ Linear 층에서 logit neuron 이.

Softmax .



다음 토큰 .

3) Self-Attention 에서 Query, Key, Value 가 각각 어떤 역할을 하는지 설명하고, 다음 scaled dot-product attention 식의 각 항이 의미하는 바를 서술하세요.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q : 현재 토큰이 "무엇을 알고 있는지"

K : 각 토큰이 "자신이 어떤 정보를 가지고 있는지"

4) Decoder 의 Masked Self-Attention 에서 미래 토큰을 보지 못하도록 masking 하는 이유를 설명하세요. Mask 가 없다면 학습과 생성 과정에서 어떤 문제가 발생할지도 함께 서술하세요.

미래의 값을 미리 알지 못하도록 ,

5) Multi-Head Attention 이 하나의 attention head 만 사용하는 것보다 유용한 이유를 설명하세요. 서로 다른 head 들이 문장에서 포착할 수 있는 관계의 예를 한 가지 이상 제시하세요.

여러 head 는
서로 다른 관계를 동시에 학습할 수 있음 ;